Gotta Go Fast

CryptoHack Write-up | Challenge 13372

Overview

The challenge description mentions generating a one-time pad forever, so I suspected the issue might be related to randomness. I checked the supplied Python source and found that the key was derived from the current timestamp. The service also lets us encrypt data we choose, which gives us a way to test the encryption without writing a solver script.

Tools used: Netcat, terminal, and CyberChef.

1. Reconnaissance

I connected to the host and port shown on the challenge page using Netcat:

```
nc socket.cryptohack.org 13372
```

The server displayed “Gotta go fast!”. I then sent two JSON requests through one connection: first, encrypt 32 zero bytes; second, return the encrypted flag.

```
printf
'{"option":"encrypt_data","input_data":"0000000000000000000000000000000000000000000000000000000000000000"}\n{"option":"get_flag"}\n' | nc socket.cryptohack.org 13372
```

The server returned these values:

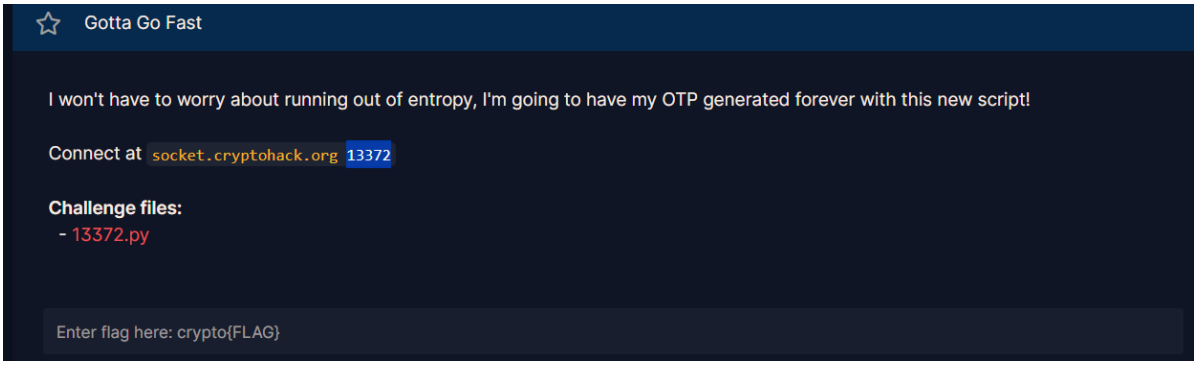
```
encrypted_data:
6d0e1428489ad58dfc1083e1df438193a8c6626bad131c228dc9ddfd32054e7

encrypted_flag:
0e7c6d583cf5aef9cc20dc87eb30f5ccdcf65289dca443ab18a9a8a2
```

The first value is the ciphertext for my chosen plaintext, and the second is the encrypted flag. I wanted to see if the first value could be used as the XOR key for the second.

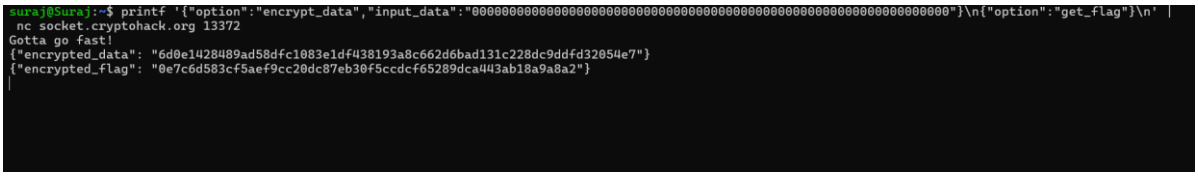
Challenge page

This screenshot shows the challenge name, target endpoint, and source file.



Terminal output

This is the terminal output from my actual run, including both responses from the service.



The main catch out here was sending both inputs at the same timestamp.

2. Source-code analysis

The key-generation function in the supplied source is:

```
def generate_key():
    current_time = int(time.time())
    key = long_to_bytes(current_time)
    return hashlib.sha256(key).digest()
```

The function takes the current Unix timestamp in seconds, converts it to bytes, and hashes it with SHA-256. The digest is 32 bytes long. SHA-256 is not the weakness by itself; the problem is that the timestamp input is predictable and only changes once per second after it is converted to an integer.

The encryption code XORs each plaintext byte with the corresponding key byte:

```
ciphertext_byte = plaintext_byte ^ key_byte
```

XOR can be reversed by applying XOR with the same key again. That makes it important to keep the key secret and unpredictable.

3. Exploitation with CyberChef

Why I used zeroes

I sent 32 zero bytes to encrypt_data on purpose. XORing any byte with zero leaves it unchanged: 0 XOR x = x. So, for that request, the ciphertext returned by the server contains the key bytes used for that request.

I then entered the encrypted flag into CyberChef. I used From Hex to convert the hexadecimal string into raw bytes, followed by XOR. For the XOR key, I pasted the encrypted_data value from the zero-plaintext request, selected HEX as the key format, and left the scheme set to Standard.

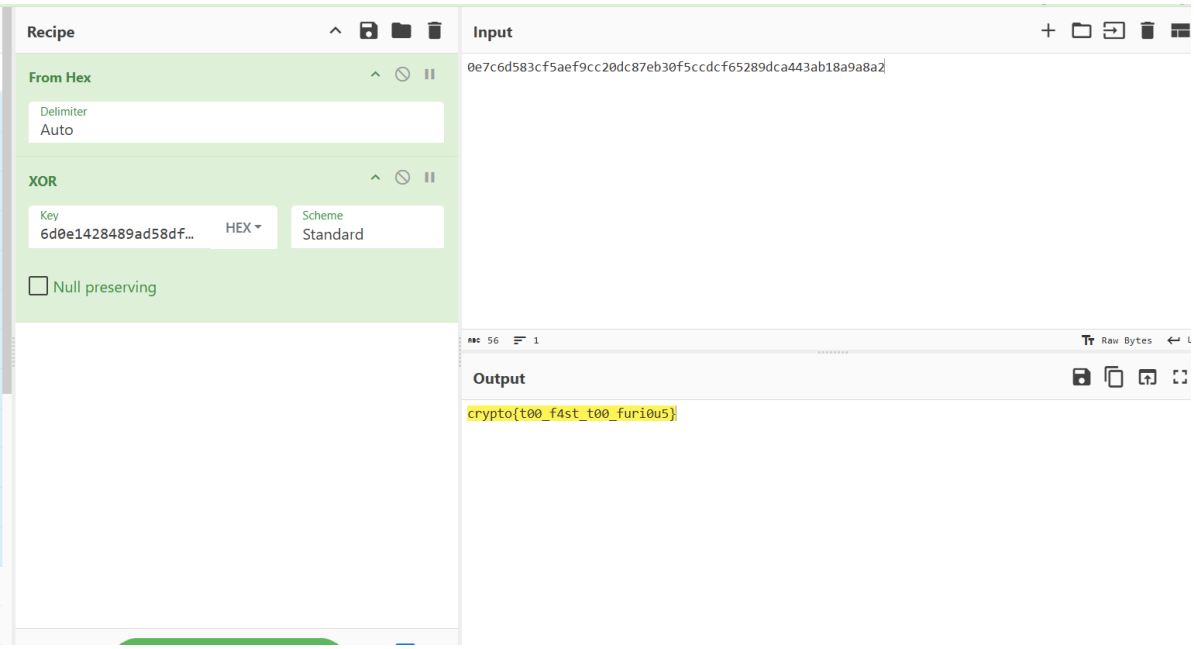
```
Input:
0e7c6d583cf5aef9cc20dc87eb30f5ccdcf65289dca443ab18a9a8a2

CyberChef recipe:
1. From Hex
2. XOR
   Key: 6d0e1428489ad58dfc1083e1df438193a8c6626bad131c228dc9ddfd32054e7
   Key format: HEX
   Scheme: Standard
```

The output was readable and matched the expected CryptoHack flag format.

CyberChef screenshot

This is the CyberChef screenshot supplied with the challenge notes. It shows the From Hex and XOR operations, the encrypted flag input, and the recovered plaintext.



4. Result and verification

The recovered flag was:

crypto{t00_f4st_t00_furi0u5}

The result begins with crypto{ and ends with }, matching the expected flag format. The terminal screenshot shows the values returned by the server, while the CyberChef screenshot shows the decryption result.

5. Impact and remediation

The root problem is that the key is derived from the current timestamp instead of a cryptographically secure random source. Hashing a predictable value does not create new entropy. The chosen-plaintext option makes the weakness easier to exploit because an all-zero input reveals the key bytes for that request.

A safer implementation should:

- Generate key material using a cryptographically secure random-number generator, such as Python's secrets module.
- Use a standard authenticated encryption scheme such as AES-GCM instead of a custom XOR construction.
- Never derive an encryption key solely from a timestamp.
- Never reuse a one-time-pad key. A true one-time pad needs uniformly random key material at least as long as the message, kept secret and used only once.

Conclusion

I solved the challenge by checking how the key was generated, using the chosen-plaintext option with zero bytes, and using CyberChef to XOR the encrypted flag with the key bytes. The main lesson is that a secure hash cannot fix predictable input.

One timing detail matters: the server generates a key separately for each request. The zero-plaintext ciphertext decrypts the flag only if both requests use the same integer timestamp. Sending the requests back-to-back can make that happen, but it is not guaranteed on every attempt.